Zero Spi Nor Flash启动系统制作指南 - Sipeed Wiki

全新 Maix 系列产品 MaixCAM 已上线, 全新 MaixPy 功能更丰富，性能更强大，软件更易用，文档更全面！ x

一、Uboot编译配置

首先规划flash分区，我测试用的flash大小为16M，型号为MX25L12873F，规划如下：

```
[第一分区1MB存放spl和uboot] 0x000000000000-0x000000100000 : "uboot"
[第二分区64KB存放dtb文件] 0x000000100000-0x000000110000 : "dtb"
[第三分区4MB存放内核] 0x000000110000-0x000000510000 : "kernel"
[剩余空间存放根文件系统] 0x000000510000-0x000001000000 : "rootfs"
```

由于目前Uboot环境变量固定存放在1MB位置之内，所有留给uboot的空间固定到flash前1MB的位置不变。每个分区的大小必须是擦除块大小的整数倍，MX25L12873F的擦除块大小是64KB。

1.1. 准备uboot

下载包含spi驱动的体验版本uboot，该驱动目前尚未合并到主线

`git clone -b v3s-spi-experimental https://github.com/Lichee-Pi/u-boot.git`

执行 `make ARCH=arm menuconfig` 打开uboot菜单配置，进入到 Device Drivers > SPI Flash Support

注意看一下自己flash的厂家名称，例如选上Macronix SPI flash support用来支持我测试用的 flash：MX25L12873F。

配置uboot默认环境变量，在文件 `include/configs/sun8i.h` 中添加默认bootcmd和bootargs的环境变量设置，注意添加的位置在 “**#include <configs/sunxi-common.h>**” 的前边。

```
#define CONFIG_USB_EHCI_SUNXI
#endif

#ifdef CONFIG_MACH_SUN8I_H3
        #define CONFIG_SUNXI_USB_PHYS   4
#elif defined CONFIG_MACH_SUN8I_A83T
        #define CONFIG_SUNXI_USB_PHYS   3
#elif defined CONFIG_MACH_SUN8I_V3S
        #define CONFIG_SUNXI_USB_PHYS   1
#else
        #define CONFIG_SUNXI_USB_PHYS   2
#endif

#define CONFIG_BOOTCOMMAND   "sf probe 0; "                          \
                             "sf read 0x41800000 0x100000 0x10000; "  \
                             "sf read 0x41000000 0x110000 0x400000; " \
                             "bootz 0x41000000 - 0x41800000"

#define CONFIG_BOOTARGS      "console=ttyS0,115200 earlyprintk panic=5 rootwait " \
                             "mtdparts=spi32766.0:1M(uboot)ro,64k(dtb)ro,4M(kernel)ro,-(rootfs) root=31:03 rw rootfstype=jffs2"

/*
 * Include common sunxi configuration where most the settings are
 */
#include <configs/sunxi-common.h>
#endif /* __CONFIG_H */
```

```
#define CONFIG_BOOTCOMMAND   "sf probe 0; "                          \
                             "sf read 0x41800000 0x100000 0x10000; "  \
                             "sf read 0x41000000 0x110000 0x400000; " \
                             "bootz 0x41000000 - 0x41800000"

#define CONFIG_BOOTARGS      "console=ttyS0,115200 earlyprintk panic=5 rootwait " \

"mtdparts=spi32766.0:1M(uboot)ro,64k(dtb)ro,4M(kernel)ro,-(rootfs) root=31:03 rw rootfstype=jffs2"
```

该环境变量通过在bootargs中增加mtdparts内核参数来告诉内核flash的分区设置信息，并通过 `root=31:03` 来告诉内核rootfs的位置。

```
sf read 0x41800000 0x100000 0x10000; //从flash0x100000（1MB）位置读取dtb放到内存0x41800000偏移处。
sf read 0x41000000 0x110000 0x400000; //从flash0x110000（1MB+64KB）位置读取dtb放到内存0x41000000偏移处。
bootz 0x41000000 （内核地址）- 0x41800000（dtb地址） 启动内核
```

1.2. 编译uboot

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j4
```

二、linux内核编译配置

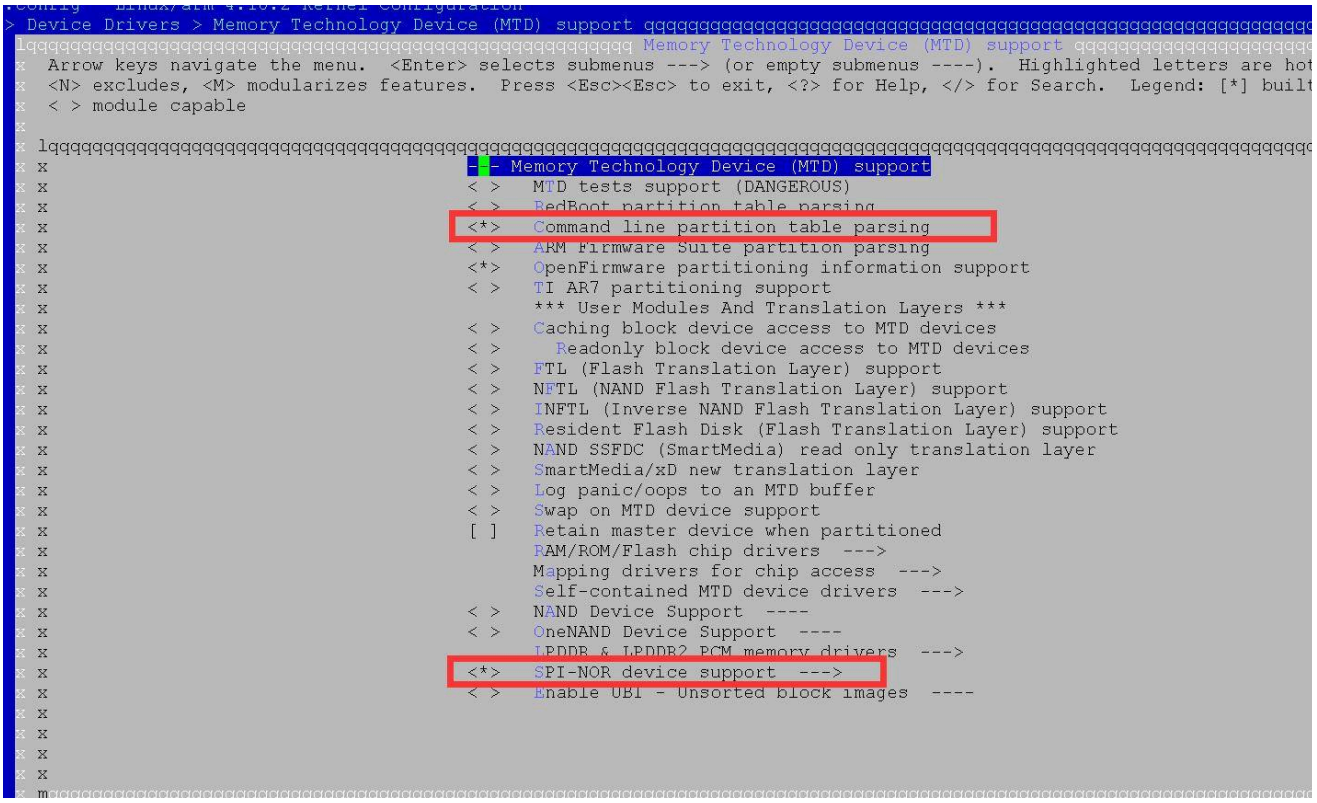
linux内核基于github上的版本 https://github.com/Lichee-Pi/linux.git

执行 `make ARCH=arm menuconfig` 打开内核菜单配置，进入到

Device Drivers → Memory Technology Device (MTD) support，确保选择上mtd的

<*> Command line partition table parsing

支持，该项目用来解析uboot传递过来的flash分区信息。



修改dts配置添加spi flash节点

```
vi arch/arm/boot/dts/sun8i-v3s-licheepi-zero.dts
```

添加spi节点配置:

```
&spi0 {
        status ="okay";

        mx25l12805d:mx25l12805d@0 {
                compatible = "jedec,spi-nor";
                reg = <0x0>;
                spi-max-frequency = <50000000>;
                #address-cells = <1>;
                #size-cells = <1>;
        };

};
```

```
                compatible = "solomon,ssd1306fb-i2c";
                reg = <0x3c>;
                solomon,width = <128>;
                solomon,height = <64>;
                reset-gpios = <&pio 1 0 GPIO_ACTIVE_HIGH>;
                solomon,page-offset = <0>;
                solomon,com-invdir;
        };

};

&spi0 {
        status ="okay";

        mx25112805d:mx25112805d@0 {
                compatible = "jedec,spi-nor";
                reg = <0x0>;
                spi-max-frequency = <50000000>;
                #address-cells = <1>;
                #size-cells = <1>;
        };

};
&uart0 {
        pinctrl-0 = <&uart0_pins_a>;
        pinctrl-names = "default";
        status = "okay";
};

&usb_otg {
        dr_mode = "otg";
        status = "okay";
};

&usbphy {
        usb0_id_det-gpio = <&pio 5 6 GPIO_ACTIVE_HIGH>;
```

添加对jffs2文件系统的支持，路径在

File systems → Miscellaneous filesystems → Journalling Flash File System v2 (JFFS2) support

```
.config - Linux/arm 4.10.2 Kernel Configuration
> File systems > Miscellaneous filesystems
  Miscellaneous filesystems
  Arrow keys navigate the menu.  <Enter> selects submenus ---> (or empty submenus ----).  Highlighted letters are
  <N> excludes, <M> modularizes features.  Press <Esc><Esc> to exit, <?> for Help, </> for Search.  Legend: [*] b
  < > module capable

        --- Miscellaneous filesystems
        < >   ORANGEFS (Powered by PVFS) support
        < >   ADFS file system support
        < >   Amiga FFS file system support
        < >   eCrypt filesystem layer support
        < >   Apple Macintosh file system support
        < >   Apple Extended HFS file system support
        < >   BeOS file system (BeFS) support (read only)
        < >   BFS file system support
        < >   EFS file system support (read only)
        <*>   Journalling Flash File System v2 (JFFS2) support
        (0)     JFFS2 debugging verbosity (0 = quiet, 2 = noisy)
        [*]     JFFS2 write-buffering support
        [ ]       Verify JFFS2 write-buffer reads
        [ ]     JFFS2 summary support
        [ ]     JFFS2 XATTR support
        [ ]     Advanced compression options for JFFS2
        < >   Compressed ROM file system support (cramfs) (OBSOLETE)
        < >   SquashFS 4.0 - Squashed file system support
        < >   FreeVxFS file system support (VERITAS VxFS(TM) compatible)
        < >   Minix file system support
        < >   SonicBlue Optimized MPEG File System support
        < >   OS/2 HPFS file system support
        < >   QNX4 file system support (read only)
        < >   QNX6 file system support (read only)
        < >   ROM file system support
```

退出菜单配置并编译内核和dts

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j4
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- dtbs
```

三、准备镜像文件

首先选择rootfs文件系统，我是用的是群朋提供的最小根文件系统rootfs-brmin.tar.gz，大小只有3M左右，下载地址在

https://pan.baidu.com/share/link?shareid=1432204556&uk=3658413294\#list/path=%2F

> Lichee Zero>zero_imager.zip

Flash支持jffs2文件系统格式，所以需要使用此该rootfs制作jffs2文件系统镜像;

下载jffs2文件系统制作工具

```
apt-get install mtd-utils
```

解压 rootfs-brmin.tar.gz

```
tar xzvf rootfs-brmin.tar.gz
```

总空间是 **16M-1M-64K-4M=0xAF0000**

```
mkfs.jffs2 -s 0x100 -e 0x10000 -p 0xAF0000 -d rootfs/ -o jffs2.img
```

页大小 **0x100 256** 字节,块大小 **0x10000 64k**

jffs2分区总空间 0xAF0000

jffs2.img是生成的文件系统镜像。

最后将uboot，dtb，kernel，rootfs打包成一个系统镜像，命令如下:

```
dd if=/dev/zero of=flashimg.bin bs=16M count=1
dd if=../ubootmainline/u-boot/u-boot-sunxi-with-spl.bin of=flashimg.bin
bs=1K conv=notrunc
dd if=../zero41y/linux-zero-4.10.y/arch/arm/boot/dts/sun8i-v3s-
licheepi-zero-dock.dtb of=flashimg.bin bs=1K seek=1024  conv=notrunc
dd if=../zero41y/linux-zero-4.10.y/arch/arm/boot/zImage of=flashimg.bin
bs=1K seek=1088  conv=notrunc
dd if=../zero_imager/jffs2/jffs2.img of=flashimg.bin  bs=1K seek=5184
conv=notrunc
```

1. 生成一个空文件，大小是16MB
2. 将uboot添加到文件开头
3. 将dtb放到1M偏移处
4. 将kernel放到1M+64K偏移处
5. 将rootfs放到1M+64K+4M偏移处

> 偏移大小是seek，单位是KB。

执行完毕后生成镜像文件flashimg.bin

四、烧写镜像

下载sunxiflash烧写工具，

```
git clone -b spi-rebase https://github.com/Icenowy/sunxi-tools.git
```

> 请选择spi-rebase分支

进入工具目录执行 * *make* *

Zero有一个usb下载模式称为fel模式。zero的启动顺序是先sd卡然后spi flash。如果sd卡启动检测失败（没有sd卡或者sd卡中没有启动ubootspl），然后spiflash也启动检测失败（没有spiflash或者spiflash中没有启动ubootspl），那么会自动进入fel模式。

所以当你的spiflash还没有启动镜像的时候你拔掉sd上电会自动进入fel模式。

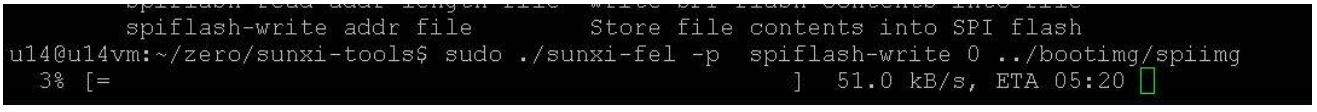
如果你的spiflash已经有了启动镜像，那么需要在sd卡中烧入一个sunxi提供的启动工具（ `dd if=fel-sdboot.sunxi of=/dev/mmcblk0 bs=1024 seek=8` ），那么插入该sd卡启动会进入fel模式；还有另一种方法，擦除spiflash内容上点也会进入fel模式。

进入fel模式后使用usb数据线连接pc和zero,进入sunxi-tool目录

> 执行 `sudo ./sunxi-fel version` 会显示硬件版本信息。
> 执行 `sudo ./sunxi-fel spiflash-info` 会显示spi flash信息。

执行如下命令烧入我们前边制作好的镜像文件

```
sudo ./sunxi-fel -p spiflash-write 0 ../bootimg/flashimg.bin
```



等待大概5分钟，烧写完成，如果没有意外重新上电zero那么就会进入linux系统了，账号是root没有密码。

```
[   10.731584] [<c04406c4>] (mtd_writev) from [<c02f5688>] (jffs2_flash_writev+0x400/0x478)
[   10.739673] [<c02f5688>] (jffs2_flash_writev) from [<c02edf84>] (jffs2_write_dnode+0xd4/0x34c)
[   10.748277] [<c02edf84>] (jffs2_write_dnode) from [<c02f270c>] (jffs2_do_setattr+0x2a8/0x500)
[   10.756795] [<c02f270c>] (jffs2_do_setattr) from [<c02f2988>] (jffs2_setattr+0x24/0x48)
[   10.764794] [<c02f2988>] (jffs2_setattr) from [<c0208dc4>] (notify_change+0x1d4/0x3bc)
[   10.772707] [<c0208dc4>] (notify_change) from [<c02193d4>] (utimes_common+0xb0/0x16c)
[   10.780529] [<c02193d4>] (utimes_common) from [<c0219540>] (do_utimes+0xb0/0x148)
[   10.788004] [<c0219540>] (do_utimes) from [<c021977c>] (SyS_futimesat+0xcc/0xfc)
[   10.795393] [<c021977c>] (SyS_futimesat) from [<c01072c0>] (ret_fast_syscall+0x0/0x3c)
[   10.803368] ---[ end trace ebba38ddd21d885f ]---
Initializing random number generator... done.
Starting network: OK

Welcome to Lichee Pi
Lichee login: root
# id
uid=0(root) gid=0(root) groups=0(root),10(wheel)
# df -k
Filesystem           1K-blocks      Used Available Use% Mounted on
/dev/root                10688      3364      7324  31% /
devtmpfs                 27560         0     27560   0% /dev
tmpfs                    28072         0     28072   0% /dev/shm
tmpfs                    28072         4     28068   0% /tmp
tmpfs                    28072        16     28056   0% /run
# df -sh
df: invalid option -- 's'
BusyBox v1.26.2 (2017-03-24 21:39:56 CST) multi-call binary.

Usage: df [-PkmhT] [FILESYSTEM]...

Print filesystem usage statistics

        -P      POSIX output format
        -k      1024-byte blocks (default)
        -m      1M-byte blocks
        -h      Human readable (e.g. 1K 243M 2G)
        -T      Print filesystem type
# df -kh
Filesystem                Size      Used Available Use% Mounted on
/dev/root                10.4M      3.3M      7.2M  31% /
devtmpfs                 26.9M         0     26.9M   0% /dev
tmpfs                    27.4M         0     27.4M   0% /dev/shm
tmpfs                    27.4M      4.0K     27.4M   0% /tmp
tmpfs                    27.4M     16.0K     27.4M   0% /run
#
```

[Qt5 移植到 licheepi zero]

[lichee 编译踩坑记录(ilichee ZERO)]